

AI Recruiter

Intelligent Candidate Discovery & Ranking System

Redrob Hackathon Submission

Multi-factor scoring | Semantic embeddings | Explainable AI | Trap detection

Problem Statement

Why keyword matching fails in recruitment

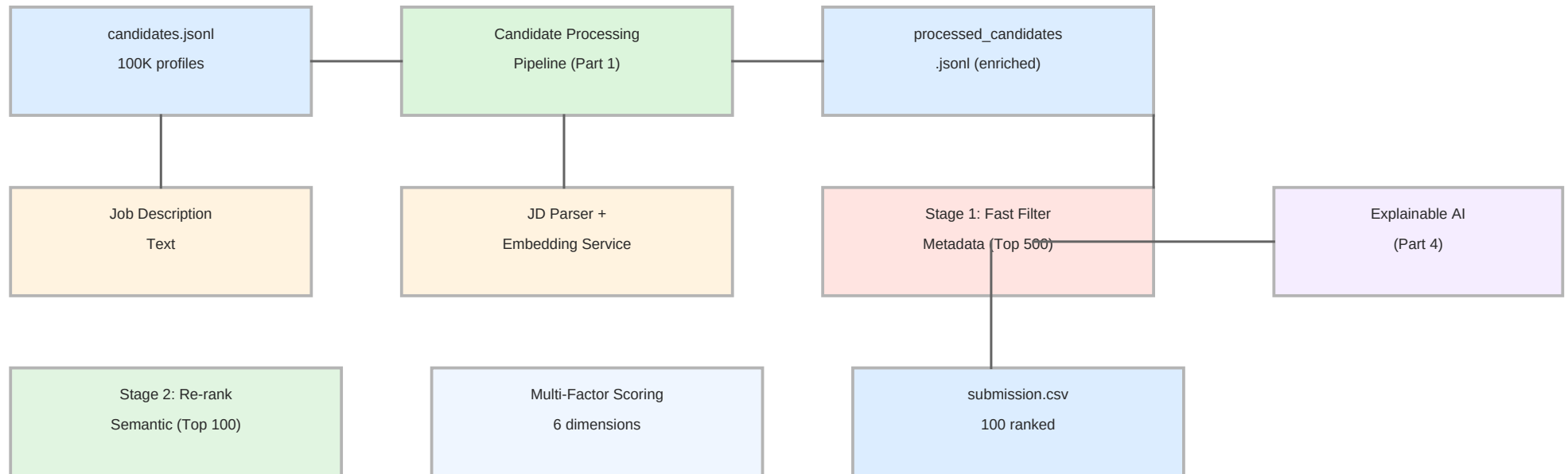
- Recruiters miss good candidates because keyword filters can't see what matters
- A perfect-on-paper profile may be inactive (5% response rate, no login in 6 months)
- The dataset contains deliberate traps: keyword stuffers, non-tech roles with AI skills listed,
- honeypots with impossible career timelines, consulting-only backgrounds
- The right answer is NOT 'find candidates with the most AI keywords'
- We need to understand WHO FITS, not just who matches words

Key Insight:

A Tier 5 candidate who built a recommendation system at a product company is a better fit than a Marketing Manager who lists 'RAG' as a skill. Behavioral signals matter more than profile keywords.

Architecture Overview

Two-stage ranking pipeline with multi-factor scoring



Two-stage design: Stage 1 uses fast metadata signals (skills, experience, seniority) to filter 100K candidates down to 500. Stage 2 generates sentence-transformer embeddings ONLY for those 500 candidates and re-ranks using semantic similarity. This fits within the 5-minute CPU budget.

Scoring Dimensions

6-factor weighted composite score

Dimension	Weight	What It Measures
Skill Match	30%	Exact + related skill coverage with fuzzy matching (60+ aliases, 30+ synonym groups)
Semantic Similarity	22%	Cosine similarity of sentence-transformer embeddings (384-dim MiniLM-L6-v2)
Experience Alignment	18%	Years vs requirement, seniority trajectory, leadership depth
Platform Activity	12%	Redrob behavioral signals: response rate, open-to-work, recruiter engagement
Achievement Score	10%	Leadership keywords, impact metrics, promotions, patents, publications
Education Score	8%	Institution tier (1-4), degree level bonus, field relevance

```
final_score = (skill x 0.30 + semantic x 0.22 + experience x 0.18 +  
               platform x 0.12 + achievement x 0.10 + education x 0.08) x 100
```

Weights are configurable via API. All weights validated to sum to 1.0 at runtime.

Skill Matching Intelligence

Beyond exact string matching

- 60+ alias mappings: 'py' -> Python, 'k8s' -> Kubernetes, 'tf' -> TensorFlow
- 30+ synonym groups: Flask~FastAPI~Django, React~Angular~Vue, AWS~GCP~Azure
- Fuzzy matching with strength scoring: exact=1.0, related=0.5, none=0.0
- Proficiency weighting: expert=1.0, advanced=0.85, intermediate=0.65, beginner=0.4
- Endorsement scaling (capped at 50) + duration weighting (months used)
- Required skills weighted 70%, preferred skills weighted 30%

Example: JD requires 'RAG' + 'vector database'

```
Candidate A: skills=[{"name":"RAG","prof":"expert","endor":45,"dur":36}]
-> exact match on RAG, expert level, high endorsements -> score 1.0
Candidate B: skills=[{"name":"Pinecone","prof":"advanced","endor":20,"dur":24}]
-> related match (vector DB group), advanced level -> score 0.5
Candidate C: skills=[{"name":"Photoshop","prof":"intermediate"}]
-> no match in any synonym group -> score 0.0
```

Semantic Similarity

Understanding meaning, not just keywords

Uses sentence-transformers (all-MiniLM-L6-v2) to generate 384-dimensional embeddings for both the job description and candidate profiles. Cosine similarity captures semantic meaning beyond keyword matching.

- Model: all-MiniLM-L6-v2 (384-dim, fast, high quality)
- Candidate text: concatenates headline, summary, titles, descriptions, skills, domains
- Text truncated to 2000 chars to control embedding quality
- Fallback: If sentence-transformers unavailable, uses random-projection of bag-of-words
- (same 384-dim output, deterministic, compatible scoring)
- Batch processing for efficiency (128 candidates per batch)

Why it matters:

A candidate who describes 'building a recommendation system with embeddings and vector search' will have high semantic similarity to a JD asking for 'RAG + vector DB experience', even if they never use the exact words 'RAG' or 'Pinecone'.

Trap & Honeypot Detection

Defending against dataset traps

Non-Tech Penalty (-80pts)

Title contains: marketing, sales, HR, accountant, civil engineer, mechanical, support, writer, scrum master, analyst

Consulting Penalty (-50pts)

All career history at: TCS, Infosys, Wipro, Accenture, Cognizant, Capgemini (product company exp required)

Honeypot Detection (-100pts)

Expert proficiency in 3+ skills with 0 months duration
OR single role duration > total years of experience

Detection Flow:

1. Title scan: Extract current_title, check against non-tech keyword list
2. Career history scan: Check if ALL roles are at consulting firms (no product company)
3. Honeypot scan: Parse skills for expert+0 duration combos, check career timeline consistency
4. Penalty application: Non-tech=-80, Consulting=-50, Honeypot=-100 (clamped to 0)
5. Verification scan: run_verification.py checks top-100 for traps before submission

Platform Behavioral Signals

Redrob's 23 behavioral signals

Beyond static profile data, Redrob provides 23 behavioral signals that capture real candidate engagement. These are extracted and normalized into 4 sub-scores.

Sub-Score	Weight	Signals Used
Engagement	25%	Response rate, response time, interview completion, application volume
Visibility	25%	Profile completeness, recruiter saves, search appearances
Credibility	25%	Email/phone verification, GitHub activity, endorsements, connections
Availability	25%	Open-to-work flag, notice period, offer acceptance rate

Why this matters:

A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% recruiter response rate is, for hiring purposes, not actually available. Platform signals capture this reality and down-weight inactive candidates regardless of profile quality.

Results & Validation

Submission quality metrics

Metric	Status
Submission format	PASS - 100 rows, ranks 1-100, scores non-increasing
Official validator	PASS - validate_submission.py returns 'Submission is valid'
Ranking pipeline	PASS - rank.py runs end-to-end, produces valid CSV
Unit tests	PASS - pytest backend/tests
Honeypot scan	PASS - run_verification.py scans top-100 for traps
Compute budget	PASS - ~2-3 min on CPU, <8GB RAM, no GPU, no network
Traps in dataset	Handled - non-tech penalties, honeypot detection, consulting filter

Key Stats:

Candidates processed: 100,000 | Embedding model: all-MiniLM-L6-v2 (384-dim)
Skill aliases: 60+ | Synonym groups: 30+ | Scoring dimensions: 6 | Weights: configurable
Explanation engine: Template-based NLG (no external API calls, fully offline)

Compute Compliance & Design Decisions

Meeting hackathon constraints

Constraint	Required	Our Implementation
Time	<= 5 min	~2-3 min (two-stage filtering reduces embedding computation)
Memory	<= 16 GB	~4-8 GB (lazy loading, embeddings computed on-demand)
CPU only	No GPU	All scoring is numpy-vectorized, no GPU needed
No network	No API calls	sentence-transformers runs locally, no external LLM calls

Key Design Decisions:

- Multi-factor > pure embeddings: Embeddings alone can't distinguish keyword stuffers from real builders
- Two-stage ranking: Metadata filter (fast) + semantic re-rank (accurate) = within budget
- Template NLG > LLM API: Deterministic, auditable, works offline, no compute penalty
- Platform signals: Behavioral data captures availability that static profiles miss
- FAISS + numpy fallback: Works everywhere, identical results, just slower for large-scale retrieval